

ARTIFICIAL INTELLIGENCE IN FINTECH / WEEK 02

Financial Data and Returns

Version 2

Start with a stock-price record. Identify what it measures, calculate a return, and explain what that return leaves out.

Follow Microsoft data from prices to returns, then test the ideas against real company events and contrasting periods.

Issue	Question	Page
01	What data did we download?	2
02	What do the price columns mean?	3
03	How much did the closing price change?	4
04	Can a stock rise after opening but still finish down?	5
05	Does the return include the dividend?	6
06	Can you sell on the ex-date and keep the dividend?	7
07	Does a stock split make an investor richer?	8
08	How much is the stock worth in TWD?	9
09	Can a USD gain become a TWD loss?	10
10	Why use log returns?	11
11	What is the return over several days?	12
12	Which average return should we use?	13
13	Can a positive average end in a loss?	14
14	Is an absent date a missing price?	15
15	Netflix fell 35%. Should we delete the observation?	16
16	Does a new ticker mean a new stock?	17

Use the examples

Read the question, predict the named result, then inspect the code and output. The discussion questions are part of each issue; there is no separate practice section.

Run the code in order in a fresh Python notebook with numpy, pandas and yfinance installed. Outputs are shown here, so you can follow the calculations without running code during class.

Stock prices are Yahoo Finance observations retrieved through yfinance, not records of personal trades. Returns exclude trading fees and taxes. Dividends are included only where stated; no trading strategy is evaluated.

Each formula is followed by its numerical example. Displayed prices and rates are rounded; code calculates with the downloaded precision, so the last shown digit may differ slightly.

ISSUE 01

What data did we download?

First identify the company, field, currency and dates. MSFT is Microsoft common stock, listed on Nasdaq; its prices here are in USD per share.

The start date is included; the end date is excluded. A daily row labels a trading date, not a trade at midnight. This request uses daily data without pre-market or after-hours data.

Before running: Count the weekdays from August 14 through August 21. Will August 22 appear?

```
import numpy as np
import pandas as pd
import yfinance as yf

msft = yf.Ticker("MSFT").history(
    start="2024-08-14", end="2024-08-22",
    interval="1d", auto_adjust=False,
    actions=True, prepost=False, keepna=True,
)
msft.index = msft.index.strftime("%Y-%m-%d")
assert msft.index.is_unique
assert msft.index.is_monotonic_increasing
close = msft["Close"]
assert close.notna().all() and (close > 0).all()
print("Dates:", ", ", ".join(msft.index))
print("Rows:", len(msft))
```

Dates: 2024-08-14, 2024-08-15, 2024-08-16, 2024-08-19, 2024-08-20, 2024-08-21

Rows: 6

The request returns 6 rows, ending on August 21. August 17-18 are a weekend. Six closing prices give five adjacent close-to-close returns, because each return needs two prices.

Asset	Frequency	Currency
MSFT	Daily	USD

auto_adjust=False keeps Close and Adj Close separate. In the following tables, each row is a date, holding interval or event; each column is a variable or calculated result.

The checks stop calculation if dates repeat or prices are missing. They do not prove that the provider's observations are correct.

Discuss

1. Why can this request produce five returns, not six? Identify the extra price needed to calculate a return ending on August 14.
2. Read the request: what would changing end to August 21 remove, and which last return would disappear?

Sources: [1], [2]. Full titles and links are on the references page.

ISSUE 02

What do the price columns mean?

Open, High, Low and Close describe different prices within a trading session. Choose the field before choosing the formula.

Open and Close are the opening and closing prices; High and Low are the session extremes. Use O, H, L and C for those fields. Volume counts shares traded, not investors or money spent. Prices below are USD per share.

Before running: For each date, must both Open and Close lie between Low and High? Check both rows, not just one.

```
fields = ["Open", "High", "Low", "Close", "Volume"]
two_days = msft.loc[["2024-08-14", "2024-08-15"], fields]
result = two_days.copy()
for field in fields[:-1]:
    result[field + " (USD)"] = result.pop(field).map("{:.2f}".format)
result["Volume (shares)"] = result.pop("Volume").map("{:, .0f}".format)
result.insert(0, "Date", result.index)
result
```

Date	Open (USD)	High (USD)	Low (USD)	Close (USD)	Volume (shares)
2024-08-14	414.80	417.72	412.45	416.86	18,267,000
2024-08-15	419.80	421.11	417.66	421.03	20,752,100

MSFT's range was USD 3.45 per share; its open-to-close increase was USD 1.23 per share. The first measures the day's range, not a return earned by buying and selling.

Volume increased from 18,267,000 to 20,752,100 shares. That measures trading activity; it does not tell us how many different people traded.

The daily bar does not reveal whether the high came before the low. It also does not establish the exact price at which your order would have traded.

Discuss

1. Using these daily bars, can you claim that buying at the low and selling later at the high was possible? Name the missing information.
2. Compare the ranges and volumes on the two dates. Does greater volume necessarily imply a wider range in these two observations? Show both calculations.

Sources: [1], [10], [13]. Full titles and links are on the references page.

ISSUE 03

How much did the closing price change?

A dollar gain and a percentage return answer different questions. Divide the price change by the starting price to express the change relative to the initial value.

For MSFT, the downloaded closes are USD 416.86 per share on August 14 and USD 421.03 on August 15. Subtract the first from the second, then divide by the first.

C_{t-1} and C_t are successive closing prices in USD per share. r_t is the simple price return, stored as a decimal. Both prices must use the same share basis, and the starting price must be positive.

$$r_t = \frac{C_t - C_{t-1}}{C_{t-1}} = \frac{C_t}{C_{t-1}} - 1$$

$$r_t = \frac{421.03 - 416.86}{416.86} \approx 0.010003 = 1.0003\%$$

The USD 4.17 increase per share is about 1% of the USD 416.86 starting price. Both inputs are closing prices, on August 14 and August 15 respectively.

Before running: Will the dollar increase and percentage increase have the same numerical value? Explain which starting value the percentage uses.

```
close_aug14 = close.loc["2024-08-14"]
close_aug15 = close.loc["2024-08-15"]
price_change = close_aug15 - close_aug14
price_return = price_change / close_aug14
print(f"Price change: USD {price_change:.2f} per share")
print(f"Price return: {price_return:.2%}")
```

Price change: USD 4.17 per share
Price return: 1.00%

The price increased by USD 4.17 per share, or 1.00%. A 1% return is not a USD 1 profit: it is a gain equal to 1% of the starting value.

This calculation covers the August 14 close to the August 15 close. It excludes the dividend, fees and taxes; it is not a record of a particular investor's realized profit.

Discuss

1. Convert the displayed percentage back to a decimal. Which starting value must you multiply it by to recover the price change, and why might rounded numbers differ slightly?
2. The result is positive. What information is still needed before saying that a student who traded MSFT on August 15 made this return?

Sources: [7], [10]. Full titles and links are on the references page.

ISSUE 04

Can a stock rise after opening but still finish down?

MSFT rose after opening on August 21, 2024, but closed below August 20's close. These statements use different starting prices.

t is August 21. O_t is that day's open; C_t is its close; C_{t-1} is August 20's close. All prices are USD per share.

Date (2024)	Open (USD)	Close (USD)
Aug 20	-	424.80
Aug 21	424.08	424.14

$$\frac{C_t - O_t}{O_t} = \frac{424.14 - 424.08}{424.08} \approx +0.014\%$$

$$\frac{C_t - C_{t-1}}{C_{t-1}} = \frac{424.14 - 424.80}{424.80} \approx -0.155\%$$

After opening, the price recovered USD 0.06 per share. It still ended USD 0.66 below the previous close.

Before running: Which starting price produces the positive return? Which produces the negative return?

```
prior = close.loc["2024-08-20"]
opening = msft.loc["2024-08-21", "Open"]
closing = close.loc["2024-08-21"]
overnight = opening / prior - 1
intraday = closing / opening - 1
full_day = closing / prior - 1
print(f"Prior close to open: {overnight:.3%}")
print(f"Open to close: {intraday:.3%}")
print(f"Close to close: {full_day:.3%}")
```

```
Prior close to open: -0.169%
Open to close: 0.014%
Close to close: -0.155%
```

The opening price was already 0.169% below the prior close. The 0.014% rise after opening did not offset that decline: the close-to-close result remained -0.155%.

Discuss

1. Explain how a USD 0.06 rise after opening can coexist with a USD 0.66 fall from the prior close.
2. Rewrite 'MSFT rose on August 21' with an explicit starting price. Do these daily observations prove an investor traded at both displayed prices?

Sources: [7], [10]. Full titles and links are on the references page.

ISSUE 05

Does the return include the dividend?

A price-only return leaves out cash distributions. Microsoft's actual August 2024 dividend lets us add one item to the price calculation.

The interval is August 14 close to August 15 close: C_{t-1} = USD 416.86/share and C_t = USD 421.03/share. D_t = USD 0.75/share is the dividend; r_t^{div} is the return including it.

Assume the holder is eligible for this dividend: the ex-date is August 15 and payment is September 12. Count the amount owed at face value; exclude taxes, fees and reinvestment.

$$r_t^{\text{div}} = \frac{C_t + D_t - C_{t-1}}{C_{t-1}}$$

$$r_t^{\text{div}} = \frac{421.03 + 0.75 - 416.86}{416.86} = \frac{4.92}{416.86} \approx 1.1803\%$$

USD 4.92/share consists of the USD 4.17 price increase plus the USD 0.75 dividend entitlement. Divide by USD 416.86/share to express this as a return.

Before running: Does adding the USD 0.75 dividend increase the return by 0.75%, or by the dividend divided by the starting price?

```
close_aug14 = close.loc["2024-08-14"]
close_aug15 = close.loc["2024-08-15"]
dividend_per_share = msft.loc["2024-08-15", "Dividends"]
price_return = close_aug15 / close_aug14 - 1
dividend_return = (
    close_aug15 - close_aug14 + dividend_per_share
) / close_aug14
result = pd.DataFrame({
    "Period (2024)": ["Aug 14-15"],
    "Price only": [f"{price_return:.2%}"],
    "Price + dividend": [f"{dividend_return:.2%}"],
})
result
```

Period (2024)	Price only	Price + dividend
Aug 14-15	1.00%	1.18%

Including the dividend changes the result from 1.00% to 1.18%. Adj Close also incorporates applicable distributions through price adjustments: do not add the dividend to its return again. Its ratio need not exactly equal this cash-dividend calculation.

Discuss

1. Which event date determines whether this dividend belongs in the holding-period calculation? Explain why the September payment date is not enough.
2. For August 14-15, choose Close or Adj Close for a price-only calculation. What is wrong with using an adjusted-price return and then adding the same dividend again?

Sources: [2], [3], [10]. Full titles and links are on the references page.

ISSUE 06

Can you sell on the ex-date and keep the dividend?

The same 100-share purchase can receive USD 75 or USD 0 from this dividend. The purchase date determines which applies.

Microsoft announced USD 0.75/share, an August 15, 2024 ex-dividend date, and September 12 payment. Below are hypothetical 100-share holdings applying that actual announcement, not observed accounts. Assume ordinary purchases and settlement; all dividend amounts are before tax.

Buy (2024)	Sell	Shares	Dividend (USD)
Aug 14	Still held	100	75
Aug 15	Still held	100	0
Aug 14	Aug 15	100	75

For an eligible holding, N is the number of shares and D_t is the dividend in USD/share. Multiplication gives the total dividend in USD.

$$\text{Dividend} = ND_t = 100 \times 0.75 = 75 \text{ USD}$$

Buy August 14 and hold: the 100 shares qualify, so USD 75 is payable September 12. August 15 establishes entitlement; it is not the cash-payment date.

Buy August 15: those 100 shares do not qualify for this dividend. Holding them until September 12 does not change the USD 0 entitlement.

Buy August 14 and sell August 15: the eligible seller keeps the USD 75 entitlement. The buyer on August 15 buys shares without this upcoming dividend.

Receiving USD 75 does not mean the entire investment earned USD 75. Price changes and costs also affect the result. These dates and rules concern this regular cash dividend.

Discuss

1. A buyer acquires 100 shares on August 15 and holds them until September 12. Why is the upcoming dividend USD 0 rather than USD 75?
2. An eligible seller sells all 100 shares on August 15. How much of this dividend remains payable, and must the seller buy the shares back before September 12?

Sources: [2], [3]. Full titles and links are on the references page.

ISSUE 07

Does a stock split make an investor richer?

A split changes the number of shares and the price per share. It does not, by itself, create investment value.

Apple announced a four-for-one split in 2020. Split-adjusted trading began on August 31. The announcement grants three additional shares for each existing share; this is an actual corporate action, not an invented portfolio.

Event	Share count	Price per share	Position value
AAPL 2020 split	Multiply by 4	Divide by 4	Unchanged

N is a pre-split share count; C is a price in USD/share on that basis. Holding market movements aside, four times as many shares at one quarter of the price represent the same value.

$$(4N)\left(\frac{C}{4}\right) = NC$$

Before running: The downloaded history uses a common split-adjusted share basis. Should its August 28-31 price return mechanically be -75%?

```
apple = yf.Ticker("AAPL").history(
    start="2020-08-28", end="2020-09-01",
    auto_adjust=False, actions=True,
)
split = apple.loc["2020-08-31", "Stock Splits"]
split_day_return = apple["Close"].iloc[-1] / apple["Close"].iloc[0] - 1
print(f"Recorded split: {split:.0f}-for-1")
print(f"Close-series return, Aug 28-31: {split_day_return:.2%}")
```

Recorded split: 4-for-1
Close-series return, Aug 28-31: 3.39%

The split record is 4-for-1, while the downloaded Close series rises 3.39%. The split identity does not predict the actual market return. Do not divide this already split-adjusted history by four again.

Numerical share-basis conversion: the August 28 history shows USD 124.8075/share on the later split basis, equivalent to USD 499.23 on the old basis. For one hypothetical old share, use $N = 1$ and $C = 499.23$. This isolates the split, not the later market-price change.

$$(4 \times 1)\left(\frac{499.23}{4}\right) = 4 \times 124.8075 = 499.23 \text{ USD}$$

Discuss

1. Why does a fourfold increase in shares not imply a fourfold increase in value? Use the split identity, not an assumed account balance.
2. Why would applying the split ratio again corrupt this return? Identify which input's share basis you would check before changing a price history.

Sources: [4], [1], [10]. Full titles and links are on the references page.

ISSUE 08

How much is the stock worth in TWD?

To express a USD price in New Taiwan dollars, multiply by the number of TWD per USD. Keep the quotation direction visible.

The Federal Reserve reports TWD 32.27 per USD for August 15, 2024. This is a New York noon reference observation, not an exchange rate synchronized with the stock close. Use it here only to translate that day's closing value, not to claim an executable trade.

C_t is the USD closing price per share. X_t is the reference rate in TWD per USD. V_t is the translated value in TWD per share.

$$V_t = C_t X_t$$

$$V_t = 421.03 \times 32.27 \approx 13,586.64 \text{ TWD/share}$$

The USD units cancel: (USD per share) multiplied by (TWD per USD) gives TWD per share.

Before running: Will the numerical TWD value be larger or smaller than the USD price? Does that numerical change mean an investment gain?

```
# Federal Reserve H.10: observed Aug 15, released Aug 19, 2024.
twd_per_usd = 32.27
close_aug15 = close.loc["2024-08-15"]
twd_value = close_aug15 * twd_per_usd
print(f"Reference rate: {twd_per_usd:.2f} TWD per USD")
print(f"Translated close: TWD {twd_value:,.2f} per share")
```

```
Reference rate: 32.27 TWD per USD
Translated close: TWD 13,586.64 per share
```

The translated value is TWD 13,586.64 per share. It expresses the same stock-price observation in another currency; it does not show a new profit.

The H.10 page was released on August 19. It is suitable for this retrospective calculation, but that publication cannot be treated as information already available on August 15.

Discuss

1. Why is multiplication appropriate for a rate labeled TWD per USD? Show the cancellation of units.
2. Can this conversion establish what a broker would have paid in TWD at the August 15 close? Identify the observation-time and transaction information that is missing.

Sources: [5], [6]. Full titles and links are on the references page.

ISSUE 09

Can a USD gain become a TWD loss?

MSFT's USD price rose from August 15 to 19, 2024, but its TWD reference value fell. Each USD bought fewer TWD at the end.

C is the close in USD/share; X is the reference rate in TWD/USD. Subscripts 0 and 1 mean August 15 and 19. R_{TWD} is the return on the translated value, as a decimal.

Date (2024)	C (USD/share)	X (TWD/USD)
Aug 15	421.03	32.27
Aug 19	421.53	32.01

$$R_{\text{TWD}} = \frac{C_1 X_1}{C_0 X_0} - 1$$

$$R_{\text{TWD}} = \frac{421.53 \times 32.01}{421.03 \times 32.27} - 1 \approx \frac{13,493.18}{13,586.64} - 1 \approx -0.688\%$$

The translated value falls from TWD 13,586.64 to TWD 13,493.18 per share. These Fed noon reference rates are not synchronized stock-close FX quotes or actual exchange proceeds.

Before running: Will converting both prices to TWD reverse the sign of the USD price return?

```
close_aug15 = close.loc["2024-08-15"]
close_aug19 = close.loc["2024-08-19"]
# Fed reference rates, TWD per USD, for these two dates.
twd_aug15 = close_aug15 * 32.27
twd_aug19 = close_aug19 * 32.01
usd_return = close_aug19 / close_aug15 - 1
fx_return = 32.01 / 32.27 - 1
twd_return = twd_aug19 / twd_aug15 - 1
print(f"USD price return: {usd_return:.3%}")
print(f"TWD per USD change: {fx_return:.3%}")
print(f"TWD value return: {twd_return:.3%}")
```

```
USD price return: 0.119%
TWD per USD change: -0.806%
TWD value return: -0.688%
```

MSFT's USD price rose 0.119%, but each USD bought 0.806% fewer TWD. The translated value fell 0.688%. The currency movement outweighed the stock-price increase.

No dividend, fees or taxes are included. The interval starts at the August 15 close. The FX releases were August 19 and 26, so they were not both available on the observation dates.

Discuss

1. Compare C_1/C_0 with X_1/X_0 using the table. Which increase or decrease explains why the TWD value falls?
2. What FX quote timing and costs would you need before calling this an executable TWD investment return?

Sources: [5], [6], [11]. Full titles and links are on the references page.

ISSUE 10

Why use log returns?

Simple returns describe proportional gains. Log returns describe the same price ratios in a form that can be added over consecutive holding intervals.

C_{t-1} and C_t are the August 14 and 15 closes in USD/share: 416.86 and 421.03. r_t is the simple return; ℓ_t is the log return. $\ln$ is natural logarithm; $\exp$ reverses it. Both returns are decimals; prices must be positive.

$$\ell_t = \ln\left(\frac{C_t}{C_{t-1}}\right) = \ln(1 + r_t), \quad r_t = e^{\ell_t} - 1$$

$$\ell_t = \ln\left(\frac{421.03}{416.86}\right) \approx 0.00995366; \quad r_t \approx e^{0.00995366} - 1 \approx 0.01000336$$

Converting back gives approximately 1.0003% simple return. The log value and simple return describe the same price ratio, not different gains.

Before running: For the positive August 14-15 price return, will the log return be slightly smaller than the simple return?

```
close_aug14 = close.loc["2024-08-14"]
close_aug15 = close.loc["2024-08-15"]
log_return = np.log(close_aug15 / close_aug14)
price_return = close_aug15 / close_aug14 - 1
result = pd.DataFrame({
    "Period (2024)": ["Aug 14-15"],
    "Simple return": [f"{price_return:.4%}"],
    "Log return": [f"{log_return:.4%}"],
})
result
```

Period (2024)	Simple return	Log return
Aug 14-15	1.0003%	0.9954%

The simple return is 1.0003%; the log return is 0.9954%. They describe the same price ratio. Their difference is not a fee or a loss. Convert the log return back before calling it a percentage gain.

Why researchers use it

The logarithm turns a product of consecutive growth factors into a sum. Researchers can add daily log returns to form weekly or monthly log returns, and express return as a change in log price. This is useful algebra; it does not make returns independent or normally distributed.

Discuss

1. Recover the simple return with $\exp(\log_return) - 1$. Why does subtracting the two displayed returns fail to measure any money lost?
2. To summarize this price movement as a percentage gain, which value would you report? Explain one different task for which the log value is convenient.

Sources: [7]. Full titles and links are on the references page.

ISSUE 11

What is the return over several days?

Each daily return starts from a different price. Multiply the price ratios to connect successive intervals.

First use MSFT's August 14, 15 and 16 closes: $C_0 = 416.86$, $C_1 = 421.03$ and $C_2 = 418.47$ USD/share. r_1 and r_2 are the two successive simple returns, as decimals.

$$r_1 = \frac{C_1 - C_0}{C_0} = \frac{C_1}{C_0} - 1 \Rightarrow 1 + r_1 = \frac{C_1}{C_0}$$

$$r_1 = \frac{421.03 - 416.86}{416.86} \approx 0.010003; \quad 1 + r_1 = \frac{421.03}{416.86} \approx 1.010003$$

The 1 represents the initial 100%. A 1.0003% rise makes the ending price 101.0003% of the starting price, or 1.010003 times as large. That is why we multiply by $1+r$, not by r .

$$(1 + r_1)(1 + r_2) = \frac{C_1}{C_0} \times \frac{C_2}{C_1} = \frac{C_2}{C_0}$$

$$\frac{421.03}{416.86} \times \frac{418.47}{421.03} = \frac{418.47}{416.86} \approx 1.003862$$

Cancel the middle price, then subtract 1: August 14-16 gained about 0.3862%. Now extend the calculation through August 21.

Before running: On August 16, the daily price return is negative. Does that necessarily make the return since August 14 negative?

```
daily_return = close.pct_change(fill_method=None).iloc[1:]
cumulative_return = (1 + daily_return).cumprod() - 1
endpoint_return = close.iloc[-1] / close.iloc[0] - 1
log_sum = np.log1p(daily_return).sum()
shown = ["2024-08-16", "2024-08-21"]
result = pd.DataFrame({"Date": shown,
    "Since Aug 14": [f"{cumulative_return[d]:.2%}" for d in shown]})
result
```

Date	Since Aug 14
2024-08-16	0.39%
2024-08-21	1.75%

Two cumulative results are shown; all five returns are used. August 16 remains +0.39% above August 14 despite that day's fall. August 14-21: $424.14/416.86 - 1$ is approximately 1.7464%.

Discuss

- Using 416.86 and 421.03, why does multiplying by r_1 give the gain, while multiplying by $1+r_1$ gives the ending price?
- Why does 421.03 cancel in the two-period product? Why is adding the two returns not exact?

Sources: [7], [10]. Full titles and links are on the references page.

ISSUE 12

Which average return should we use?

Use the arithmetic mean to describe the average observed one-period return. Use the geometric mean to find a constant per-period return that compounds to the same ending value.

There are $n = 5$ close-to-close returns in the same August 14-21 sample. The bar over r denotes their arithmetic mean; g is their geometric mean. R is the full-period return from Issue 11. All rates are decimals in the formulas.

$$\bar{r} = \frac{r_1 + \dots + r_n}{n}, \quad g = (1 + R)^{1/n} - 1$$

For g , take the fifth root of $1+R$, then subtract 1. This reverses multiplying the same growth factor five times; it is not a forecast.

Substitute all five returns as decimals; $R = 424.14/416.86 - 1$ is approximately 0.017464.

$$\bar{r} = \frac{0.010003 - 0.006080 + 0.007312 + 0.007757 - 0.001554}{5} \approx 0.003488$$

$$g = (1 + 0.017464)^{1/5} - 1 \approx 0.003469$$

Before running: Which of these averages, repeated for five intervals, must reproduce the observed full-period return?

```
n = len(daily_return)
arithmetic_mean = daily_return.mean()
geometric_mean = (1 + endpoint_return)**(1 / n) - 1
result = pd.DataFrame({
    "Period (2024)": ["Aug 14-21"],
    "Arithmetic mean": [f"{arithmetic_mean:.4%}"],
    "Geometric mean": [f"{geometric_mean:.4%}"],
})
result
```

Period (2024)	Arithmetic mean	Geometric mean
Aug 14-21	0.3488%	0.3469%

The averages are 0.3488% and 0.3469% per close-to-close interval. Compounding the geometric mean five times gives 1.7464%, the observed full-period price return.

Here 'per interval' means one adjacent pair of trading-date closes. A Friday-to-Monday interval includes the weekend; it is not one calendar day.

Discuss

1. Calculate $(1 + \text{geometric_mean})^5 - 1$ and compare it with Issue 11. What exactly does their agreement establish?
2. A report calls the arithmetic mean the 'constant daily growth rate.' Rewrite that statement using the appropriate average, without claiming a future return.

Sources: [7]. Full titles and links are on the references page.

ISSUE 13

Can a positive average end in a loss?

A positive arithmetic mean does not guarantee a positive compounded return. MSFT on March 6-10, 2020 provides a real counterexample.

This period is selected to demonstrate the distinction, not to estimate how often it occurs. Use the closes on March 6, 9 and 10: three prices produce two close-to-close returns.

For two simple returns r_1 and r_2 , the cumulative return R includes their product as well as their sum. Opposite signs make that product negative.

$$R = (1 + r_1)(1 + r_2) - 1 = r_1 + r_2 + r_1r_2$$

Before running: The second return is positive. Does adding the two returns establish that the ending close exceeds the first close?

```
march = yf.Ticker("MSFT").history(
    start="2020-03-06", end="2020-03-11", auto_adjust=False,
)
march_close = march["Close"]
march_returns = march_close.pct_change(fill_method=None).iloc[1:]
march_mean = march_returns.mean()
march_total = march_close.iloc[-1] / march_close.iloc[0] - 1
result = pd.DataFrame({
    "Return ending": march_returns.index.strftime("%Y-%m-%d"),
    "Simple return": march_returns.map(lambda r: f"{r:.4%}"),
})
result
```

Return ending	Simple return
2020-03-09	-6.7773%
2020-03-10	6.8384%

The two returns are -6.7773% and 6.8384%. Their arithmetic mean is positive, 0.0306% per interval, yet the full-period price return is -0.4023%.

$$R \approx (1 - 0.06777255)(1 + 0.06838404) - 1 \approx -0.004023 = -0.4023\%$$

$$R \approx (-0.06777255) + (0.06838404) + (-0.06777255) \times (0.06838404)$$

The rebound applies to the lower value left after the loss. A percentage rise measured from that lower base can exceed the earlier percentage decline without fully recovering the starting value. The product term shows exactly why simply adding rates fails.

Discuss

1. Calculate r_1r_2 and add it to r_1+r_2 . Which term changes the sign of the full-period result in this example?
2. After the March 9 decline, calculate the return needed to recover the March 6 close. Compare it with the actual March 10 return; do not round before comparing.

Sources: [7], [10]. Full titles and links are on the references page.

ISSUE 14

Is an absent date a missing price?

No row on a non-trading day is different from a missing price on a trading day. Do not turn either situation into a measured zero return.

MSFT's August 16 and 19 observations are Friday and Monday. August 17-18 have no regular-session daily bars. Nasdaq's regular session was 09:30-16:00 New York time on these trading dates; detailed time-zone conversion is not needed for this calculation.

Before running: Which two calendar dates should have no regular-session row? Does their absence mean the download lost a trading day's price?

```
result = pd.DataFrame({
    "Date": ["2024-08-16", "2024-08-17", "2024-08-18", "2024-08-19"],
    "Day": ["Friday", "Saturday", "Sunday", "Monday"],
})
result["Row returned"] = result["Date"].isin(close.index).map({True: "Yes", False: "No"})
result
```

Date	Day	Row returned
2024-08-16	Friday	Yes
2024-08-17	Saturday	No
2024-08-18	Sunday	No
2024-08-19	Monday	Yes

The Saturday and Sunday rows are absent from the source, not deleted for this example. The return ending on Monday uses Friday's close. Its interval spans three calendar days, but only one adjacent close-to-close step.

Before running: Can all six prices be present while the first return is still missing?

```
returns_with_first = close.pct_change(fill_method=None)
print("Missing Close values:", int(close.isna().sum()))
print("Missing return values:", int(returns_with_first.isna().sum()))
print("First return date:", returns_with_first.index[0])
```

```
Missing Close values: 0
Missing return values: 1
First return date: 2024-08-14
```

There are no missing Close values and one missing return. The return ending August 14 has no starting close inside this sample. It is unknown, not zero; use an earlier observation if you need that return.

Discuss

1. Before filling a gap on a weekday, what would you check in the exchange calendar and source request? Why is a weekday alone not proof that the market was open?
2. Why does filling the first missing return with zero change the average's meaning? State which missing starting observation would resolve the problem instead.

Sources: [1], [8]. Full titles and links are on the references page.

ISSUE 15

Netflix fell 35%. Should we delete the observation?

A large fall is a reason to check the price, not proof that the price is wrong. Deleting a genuine loss removes part of the risk we want to study.

Use Netflix's April 19-20, 2022 closing prices. Both downloaded prices use the same later split-adjusted share basis; they are not the unadjusted dollar quotes displayed in 2022. The return is unchanged by scaling both prices by the same factor.

Before running: A proposed rule deletes every daily return below -20%. Would it remove this observation?

```
netflix = yf.Ticker("NFLX").history(
    start="2022-04-19", end="2022-04-21",
    auto_adjust=False, actions=True, keepna=True,
)
nflx_close = netflix["Close"]
assert len(nflx_close) == 2 and nflx_close.notna().all()
nflx_return = nflx_close.iloc[-1] / nflx_close.iloc[0] - 1
print(f"NFLX price return, Apr 19-20: {nflx_return:.2%}")
print(nflx_close.round(3).to_string())
```

NFLX price return, Apr 19-20: -35.12%

Date

2022-04-19 00:00:00-04:00 34.861

2022-04-20 00:00:00-04:00 22.619

C_{t-1} and C_t are these two closes in USD/share, on the same share basis. r_t is the simple return, as a decimal.

$$r_t = \frac{C_t}{C_{t-1}} - 1 = \frac{22.619}{34.861} - 1 \approx -0.3512 = -35.12\%$$

The observed -35.12% return is below -20%, so that proposed rule would delete it. A threshold tells us the fall is large; it does not tell us the recorded price is wrong.

What you establish	Action
Large fall; not yet checked	Keep and investigate
Fall corroborated; no error found	Do not delete just for its size
A specific price or date error	Correct using source evidence

Check the same security, dates and share basis against another reliable price source. Netflix's dated announcement gives event context, not independent price verification. If the loss is real, deleting it can understate downside risk.

Discuss

1. The return is -35.12%. Does the -20% deletion rule prove an error? What independent price evidence would justify a correction?
2. If this is a genuine fall, what information disappears when it is deleted? Would you retain it when measuring historical downside risk, and why?

Sources: [1], [9]. Full titles and links are on the references page.

ISSUE 16

Does a new ticker mean a new stock?

Before joining price histories, establish whether a ticker change represents the same security. A different label does not necessarily mean a different investment.

Meta's May 31, 2022 SEC-filed announcement states that its Class A common stock would switch from FB to META before the June 9 market open. Nasdaq listing and the security's CUSIP identifier would remain unchanged.

Date	Ticker	Company	Share class	Market
2022-06-08	FB	Meta Platforms	Class A	Nasdaq
2022-06-09	META	Meta Platforms	Class A	Nasdaq

Read the record before joining prices

These are two dated labels for the same Class A security, based on the announcement. Shareholders did not need to exchange their shares merely because the ticker changed. The announcement supplies identity evidence, not a new price or a guaranteed return.

A provider may show today's ticker on the entire historical series. That can be convenient for retrieval, but it does not establish which ticker was used on a past trading date. Check the provider's history rather than treating a failed FB request as proof that earlier prices do not exist.

Discuss

1. Which facts in the announcement support joining the FB and META histories? Explain why ticker text alone would miss this connection.
2. What checks are still necessary before computing the June 8-9 return from two files? State the dates, price field, units and share-basis conditions; the announcement alone does not supply them.

Sources: [12]. Full titles and links are on the references page.

REFERENCES & REPRODUCTION

Sources and calculation notes

Tables show selected observations, event facts and calculated results. The two-day OHLCV excerpt supports field interpretation; full downloaded files are not included. FX observations are transcribed from the Federal Reserve's dated releases. Rows identify samples or events; columns identify variables.

- [1] [yfinance: historical-data parameters](#)
- [2] [Microsoft: dividend announced June 12, 2024](#)
- [3] [Investor.gov: ex-dividend dates](#)
- [4] [Apple: four-for-one split announcement](#)
- [5] [Federal Reserve: H.10, August 19, 2024](#)
- [6] [FRED: DEXTAUS units and observation time](#)
- [7] [Zivot: Return Calculations](#)
- [8] [Nasdaq: market hours and calendar](#)
- [9] [Netflix: Q1 2022 shareholder letter](#)
- [10] [Yahoo Finance: MSFT historical data](#)
- [11] [Federal Reserve: H.10, August 26, 2024](#)
- [12] [Meta: ticker change, SEC exhibit dated May 31, 2022](#)
- [13] [Nasdaq glossary: volume](#)

Reproduce the examples

Run Issue 1 first, then continue in order. Each last table expression displays in a Python notebook; in a plain Python script, use `print(result)`. Install dependencies only if they are missing: `python -m pip install numpy pandas yfinance`.

Executed 2026-09-14 (UTC). Python 3.12.9; numpy 2.2.6; pandas 2.2.3; yfinance 1.2.0. The printed summaries and tables were generated by those displayed code blocks. Full-precision values are used before rounding for display.

Historical provider observations can be revised. Internet access and provider availability are required to repeat downloads. Yahoo's historical-page web preview was unavailable during this build; downloads through yfinance succeeded. Price-field conventions follow the course's prior source check; the exact adjustment algorithm is not re-derived here.

This short sample teaches calculation, not expected returns or a trading recommendation. Reading the log-return identity does not establish a distributional model. Calendar checks do not establish that every observed price is correct.